ENERGIAIOT

Monitoramento e Controle de Energia de Baixa Tensão

CP12 — Proposta de upgrade do projeto N1 para N3
Versão de trabalho 1.1 — 02/11/2026

ENERGIAIOT

Monitoramento e Controle de Energia de Baixa Tensão

CP12 — Proposta de upgrade do projeto N1 para N3
Versão de trabalho 1.1 — 02/11/2026

Equipe	Integrantes	Repositório
EnergiaIOT	Bruno Lima Costa; Caio Rosa; Iago Koch; João Francisco da Silva; Pedro Henrique Israel; William Vodzinsky	https://github.com/caiorosa31/iot

Status deste documento: proposta técnica em planejamento. As escolhas confirmadas pelo grupo estão registradas; valores elétricos, modelo exato dos componentes, testes e implementação permanecem pendentes quando indicado.

1. Ponto de partida: projeto da N1

Item	Descrição
Problema e usuário	Monitorar variáveis elétricas de uma carga de baixa tensão de forma simples, remota e segura. Público-alvo pretendido: pessoas e organizações que utilizam equipamentos de baixa tensão e precisam acompanhar consumo, identificar comportamento anormal e controlar cargas.
O que funcionou na N1	A comunicação entre o hardware e a interface recebia informações. O repositório e o histórico da N1 descrevem ESP32/Arduino, INA219, MQTT/Mosquitto e uma interface HTML com demonstração de acionamento por LED.
Limitações conhecidas	Problemas de fixação/contato do sensor na protoboard prejudicaram a medição e a demonstração. A solução ainda não possuía backend e banco de dados integrados.
Proposta de valor	Oferecer monitoramento remoto de equipamentos de baixa tensão, histórico e análise de consumo, alertas de anomalias e acionamento remoto/automático. O objetivo é reduzir desperdício, facilitar a supervisão e antecipar falhas ou condições anormais.
Troca de projeto	Não. O grupo mantém o projeto defendido na N1.

Escopo de segurança: o protótipo e a proposta desta documentação operam exclusivamente com cargas isoladas de baixa tensão em corrente contínua. Não haverá ligação direta à rede elétrica residencial ou industrial.

2. Upgrade por camada

Camada	Como era na N1	Como será na N3 (proposta)
Hardware	ESP32/Arduino e um sensor INA219; demonstração com LED.	Dois dispositivos físicos independentes, cada um com ESP32, INA219 e ventilador DC de 12 V igual. Cada dispositivo terá um deviceId. Um MOSFET adequado fará o acionamento físico da carga.
Firmware	C++ no microcontrolador.	Manter C++; implementar leitura periódica não bloqueante, validação, regra local de segurança, estado seguro, reconexão, configuração persistente, logs e OTA.
Comunicação MQTT	Mosquitto; comunicação básica entre hardware e interface.	Manter Mosquitto; contrato JSON versionado, tópicos separados para telemetria, estado, comando, confirmação e disponibilidade; Last Will, QoS/retain justificados.

Camada	Como era na N1	Como será na N3 (proposta)
Backend	Não existia como camada própria integrada.	Python com FastAPI para consumir MQTT, validar mensagens, persistir dados, expor API e ser o único publicador de comandos.
Banco de dados	Não existia integração persistente.	PostgreSQL para medições, dispositivos, configurações, usuários, alertas e ciclo de comandos.
Interface	Página HTML de demonstração.	Aplicação Vue.js com autenticação, visão geral, dados atuais, gráficos históricos, alertas, configurações e controle dos dispositivos.
Decisão/regra	A lógica remota ainda não estava definida/documentada.	Regra local de segurança no ESP32 e regra remota no backend: comparar consumo com limite configurado e solicitar desligamento quando a condição persistir.

3. Conferência contra o piso R1–R18

Requisito	Como o projeto pretende atender
R1 — Hardware	Dois pontos de medição, um em cada dispositivo ESP32 + INA219, e acionamento físico de ventilador DC 12 V por MOSFET. Dimensionamento final depende da corrente nominal dos ventiladores e dos datasheets.
R2 — Firmware	C++ com leitura periódica não bloqueante, validação de faixa, regra local de desligamento por condição anormal, saída desligada no boot, reconexão Wi-Fi/MQTT, configuração persistente em NVS, deviceId, timestamp e logs Serial.
R3 — Contrato MQTT	Criar CONTRATOS.md versionado com tópicos e JSON contendo schemaVersion, deviceId, timestamp e unidades; comando com commandId; confirmação/resultado; Last Will para disponibilidade; justificar QoS e retain.
R4 — Backend	FastAPI consome MQTT, valida payloads, registra mensagens inválidas, persiste medições/estados e oferece API para painel e comandos. Backend será o único publicador de comandos.
R5 — Banco	PostgreSQL com histórico de medições e comandos, além de dispositivos, configurações, usuários e alertas. Schema versionado por migrações.
R6 — Interface	Vue.js exibirá medições com unidade, estado atual, histórico/gráficos, último contato, online/offline, alertas e ações. Só mostrará execução após confirmação.
R7 — Ciclo de comando	Estados distintos: enviado/registrado pelo backend, recebido e aceito/rejeitado pelo ESP32, executado fisicamente e confirmado por publicação de estado/ack refletida na interface.
R8 — Decisão remota	Backend avaliará medições persistidas e limite configurável. Se o limite for excedido por condição definida, registra alerta e envia comando de desligamento pelo ciclo R7.
R9 — Segurança básica	Login na interface e autenticação nas rotas API; autorização obrigatória para comandos; segredos em variáveis de ambiente e nunca no repositório.
R10 — Falhas	Plano de testes: queda Wi-Fi, broker indisponível, sensor desconectado/leitura inválida e comando inválido ou duplicado. Resultado observado e evidências serão preenchidos após execução.
R11 — Entrega executável	README com pré-requisitos, .env.example, instruções para iniciar serviços, configurar banco, gravar firmware e executar testes; release/tag e commits de todos os integrantes.
R12 — Orquestração	Docker Compose para subir Mosquitto, backend e PostgreSQL com um comando; frontend poderá integrar o mesmo compose. Segredos por .env.
R13 — Múltiplos dispositivos	Dois ESP32 físicos, cada um com deviceId próprio, INA219 e ventilador DC 12 V. Backend e interface distinguirão os dispositivos.
R14 — Painel analítico	Gráficos e indicador agregado calculados a partir de medições reais persistidas, como média móvel de potência e energia acumulada por período.
R15 — Notificação externa	Telegram como primeira opção para alertas externos. WhatsApp poderá ser avaliado como evolução, mas não será assumido como implementado.
R16 — Segurança avançada	Perfis com permissões diferentes: administrador (gestão de usuários/dispositivos/configurações) e operador (consulta e comandos permitidos). Implementar autorização efetiva no backend.
R17 — OTA	Planejar atualização remota de firmware ESP32 sem USB e demonstrar atualização real durante a defesa. Ainda não implementado.
R18 — Testes e CI	Pytest para testes automatizados de validação de contrato e regras do backend; GitHub Actions executará testes a cada push e pull request.

4. Arquitetura-alvo

A arquitetura separa medição/atuação física, transporte MQTT, processamento e decisão no backend, persistência e interface. Sensores e atuadores não se comunicam diretamente entre si: o backend centraliza regras remotas e comandos.

4.1 Fluxo de dados

Cada INA219 fornece tensão e corrente ao ESP32. O firmware valida as leituras, calcula potência quando aplicável e publica telemetria com timestamp e unidades. O backend valida a mensagem, persiste no PostgreSQL e disponibiliza os dados pela API ao Vue.js.

4.2 Fluxo de comando

O usuário solicita uma ação no Vue.js. A API autentica e autoriza, registra o comando e o backend publica no tópico MQTT. O ESP32 valida o comando, responde aceitando ou rejeitando, aciona o MOSFET quando permitido e publica o estado/resultado. A interface atualiza o status somente após confirmação.

4.3 Identificação dos dispositivos

Dispositivo	Identidade proposta	Hardware
EnergiaIOT 01	energiaiot-01	ESP32 + INA219 + ventilador DC 12 V + estágio MOSFET
EnergiaIOT 02	energiaiot-02	ESP32 + INA219 + ventilador DC 12 V + estágio MOSFET

5. Hardware: desenho e dimensionamento

Configuração escolhida pelo grupo: dois ventiladores DC iguais de 12 V, cada um monitorado e controlado por um ESP32 independente. Os valores numéricos de corrente e potência ainda não podem ser preenchidos porque o modelo do ventilador não foi escolhido/comprado.

5.1 Verificação elétrica (a concluir com datasheets)

Verificação	Valor / critério de projeto	Como será garantido
Tensão da carga	12 V DC nominal; confirmar tolerância no datasheet do ventilador.	Fonte regulada de 12 V dimensionada após conhecer corrente nominal e partida.
Corrente estimada do atuador	Pendente: corrente nominal e corrente de partida do modelo escolhido.	Selecionar MOSFET, proteção, fios e fonte com margem acima dos valores do datasheet.
Alimentação do ESP32	Conforme placa ESP32 escolhida; preferir conversor buck 12 V → 5 V regulado ou alimentação apropriada documentada.	Verificar corrente disponível e limites da placa; não aplicar 12 V diretamente ao pino 3V3.
GND comum	Referência comum entre ESP32, INA219 e circuito de acionamento quando exigido pelo esquema.	Desenhar retorno de corrente e conexões de terra no esquema final.
Tensão nos GPIOs	Somente níveis lógicos compatíveis com ESP32 (3,3 V).	INA219 alimentado/ligado em lógica compatível; nunca aplicar 5 V diretamente ao GPIO.
Proteção da carga indutiva	Proteção de surto/roda livre conforme datasheet do ventilador e do estágio de acionamento.	Prever diodo ou proteção indicada pelo fabricante; confirmar se a carga BLDC possui proteção interna.
Estado seguro no boot	Ventilador desligado.	Gate do MOSFET com resistor pull-down e saída inicializada em OFF.

Topologia proposta por dispositivo: fonte regulada de 12 V alimenta o ventilador; INA219 fica em série no caminho de alimentação da carga, respeitando polaridade e limite do módulo; MOSFET canal N logic-level comanda a carga; ESP32 é alimentado por conversor apropriado. O circuito final deverá ser conferido com os datasheets e o módulo INA219 efetivamente comprado.

5.2 BOM — materiais

Já informado pelo grupo	A comprar / confirmar
1 ESP32 (quantidade exata e modelo a confirmar); 1 módulo INA219; componentes da montagem N1 (inventário precisa ser conferido).	1 ESP32 adicional; 1 INA219 adicional; 2 ventiladores DC 12 V iguais; 2 fontes 12 V dimensionadas (ou fonte única com canais/proteção adequados); conversores buck para ESP32 se necessários; 2 MOSFETs logic-level adequados; resistores de gate/pull-down; proteção de carga; bornes, fios, placa/perfboard e caixa.

Mapa de pinos: definir após confirmar o modelo da placa. Sugestão inicial a validar: I²C SDA GPIO 21 e SCL GPIO 22; comando de gate GPIO 25 (dispositivo 01) e GPIO 26 (dispositivo 02). Não utilizar GPIO 6–11; evitar pinos de strapping para atuadores. Confirmar pinagem real e conflitos antes de congelar o esquema.

5.3 Fluxograma do firmware

• Boot: saída física em OFF, carregar deviceId e configuração persistente.

• Medição: leitura periódica não bloqueante; validar faixa plausível e valores-sentinela.

• Regra local: se leitura inválida ou limite local de segurança for ultrapassado, desligar carga e registrar falha, mesmo sem rede.

• Modo manual: aceitar comando autorizado/validado, aplicar tempo máximo configurado e retornar ao automático ao expirar.

• Reconexão: tentar Wi-Fi e broker novamente; após reconectar, refazer assinaturas e publicar estado.

• Persistência: salvar ao menos limite de segurança/configuração remota na NVS para sobreviver a reinicialização.

6. Contrato MQTT — proposta v0 para congelar como v1

Prefixo: energiaiot/v1/devices/{deviceId}/. O backend é o único componente autorizado a publicar comandos. Os ESP32 publicam telemetria, estado, confirmação e disponibilidade.

Tópico	Direção	Uso
.../telemetry	ESP32 → broker → backend	Medições periódicas: tensão, corrente, potência e energia se cálculo/integração estiver implementado.
.../state	ESP32 → broker → backend	Estado atual da carga, modo, configuração ativa e resultado físico confirmado.
.../command	Backend → ESP32	Comando remoto: ligar, desligar ou alterar configuração.
.../ack	ESP32 → backend	Recebimento, aceitação/rejeição e resultado do comando com commandId.
.../availability	ESP32 → broker/backend	online/offline; Last Will publica offline retido em desconexão inesperada.

Exemplo de telemetria (valores ilustrativos, não medidos):

```
{
  "schemaVersion": 1,
  "deviceId": "energiaiot-01",
  "timestamp": "2026-11-02T12:00:00Z",
  "measurements": {
    "voltage": {"value": 12.1, "unit": "V"},
    "current": {"value": 0.42, "unit": "A"},
    "power": {"value": 5.08, "unit": "W"}
  },
  "quality": "valid"
}
```

Exemplo de comando:

```
{
  "schemaVersion": 1,
  "deviceId": "energiaiot-01",
  "commandId": "UUID",
  "timestamp": "2026-11-02T12:01:00Z",
  "action": "set_power",
  "value": "OFF",
  "timeoutSeconds": 0
}
```

Exemplo de confirmação:

```
{
  "schemaVersion": 1,
  "deviceId": "energiaiot-01",
  "commandId": "UUID",
  "timestamp": "2026-11-02T12:01:01Z",
  "status": "accepted",
  "result": "pending_execution",
  "reason": null
}
```

Política inicial a validar: QoS 1 para comandos e confirmações; QoS 0 ou 1 para telemetria conforme estabilidade da rede e necessidade de entrega; retain para disponibilidade/estado atual quando apropriado, não para comandos não idempotentes. Last Will retido com availability=offline. A política final deve ser registrada no CONTRATOS.md.

7. Tecnologias pretendidas

Camada	Tecnologia	Justificativa
Backend	Python + FastAPI	API REST, validação de payloads, integração MQTT e regras de negócio com desenvolvimento enxuto.
Banco	PostgreSQL	Banco relacional para medições, dispositivos, configurações, usuários, alertas e comandos; migrações versionadas.
Interface	Vue.js	Aplicação web organizada por componentes, com painel responsivo e gráficos.
Broker MQTT	Eclipse Mosquitto	Broker leve e compatível com ESP32; permite tópicos, QoS, retain, autenticação e Last Will.
Orquestração	Docker Compose	Subir broker, backend e PostgreSQL com um comando e configuração por .env.
Testes/CI	Pytest + GitHub Actions	Automatizar validação de contrato/regras e executar testes em push e pull request.
Notificação	Telegram Bot API	Canal externo inicial para alertas; WhatsApp fica como possível evolução após avaliar integração oficial.

7.1 Modelo inicial do banco

Tabela	Campos principais propostos
devices	id, device_id (único), nome, localização lógica, modelo, status, last_seen_at, created_at
measurements	id, device_id, grandeza, valor, unidade, timestamp, quality
commands	command_id, device_id, usuário/origem, ação, payload, enviado_em, recebido_em, resultado, confirmado_em
device_configs	device_id, limite_potencia_w, timeout_manual_s, versão, atualizado_em
users	id, nome, email, senha_hash, perfil, ativo
alerts	id, device_id, tipo, severidade, mensagem, criado_em, resolvido_em

8. Cronograma, riscos e divisão

8.1 Cronograma até a N3

Período/data do plano	Marco	Entrega prevista
Até 03/11/2026	Revisão N2 e fechamento do desenho	Revisar CP12, confirmar datasheets, fechar requisitos, tópicos MQTT e diagramas.
10/11/2026	Unidade 3 — etapa 1	Comprar/validar componentes, montar um dispositivo, validar medições e acionamento.
17/11/2026	Unidade 3 — etapa 2	Montar segundo dispositivo; integrar MQTT, backend, PostgreSQL e API.
24/11/2026	Unidade 3 — etapa 3	Finalizar Vue, regras, notificações, Docker Compose, segurança, OTA e testes.
01/12/2026	N3 — defesa	Demonstrar dois dispositivos físicos, painel, histórico, ciclo de comando, falhas e evidências.

As datas seguem o calendário descrito nos documentos da disciplina; o grupo deve confirmar com o professor eventuais atualizações de prazo.

8.2 Riscos e mitigação

Risco	Mitigação
Corrente do ventilador exceder limite do INA219/MOSFET/fonte	Escolher o ventilador após conferir corrente nominal e partida; conferir datasheets e dimensionar com margem.
Medições instáveis por montagem/protoboard	Usar montagem firme, bornes/placa adequada, cabos curtos e fixação mecânica.
Queda de Wi-Fi ou broker	Regra local de segurança independente da rede; reconexão automática e Last Will.

Risco	Mitigação
Comando duplicado ou atrasado	commandId único, idempotência, validação de timeout e registro de resultado.
Atraso na integração de camadas	Definir contratos/API cedo, integrar em etapas e testar cada camada isoladamente.
OTA ou WhatsApp consumir tempo excessivo	Priorizar piso R1–R14; implementar Telegram e perfis; reservar tempo específico para OTA e CI, sem declarar requisito atendido antes da demonstração.

8.3 Divisão de responsabilidades

Integrantes	Responsabilidade principal
Bruno Lima Costa e Caio Rosa	Hardware, aquisição/validação de componentes, montagem e testes elétricos.
Iago Koch e João Francisco da Silva	Firmware ESP32, leitura INA219, estados de segurança, reconexão e MQTT.
Pedro Henrique Israel e William Vodzinsky	Backend FastAPI, PostgreSQL, API, Vue.js, integração e painel.
Todos	Revisão do contrato, integração, testes R10, documentação, commits individuais e preparação da defesa.

A divisão define responsáveis principais, não exclusividade: todos os integrantes devem compreender e contribuir com o sistema completo.

9. Repositório GitHub

Repositório informado pelo grupo: https://github.com/caiorosa31/iot

Estrutura recomendada:

```
iot/
  firmware/
    device/
  backend/
  frontend/
  infra/
    mosquitto/
    compose.yaml
  docs/
    arquitetura.png
    fluxo_dados.png
    fluxo_comando.png
    firmware_flow.png
    esquema_eletrico/
  CONTRATOS.md
  README.md
  .env.example
  .gitignore
```

Para R11 e avaliação do repositório: README com pré-requisitos e instalação, .env.example sem segredos reais, instruções de execução e gravação do firmware, release/tag identificada, diagramas versionados e commits de cada integrante. Hash do commit de entrega e usuários GitHub individuais ainda precisam ser registrados.

Pendências obrigatórias antes de congelar o projeto

- Identificar modelo exato do ESP32 e do INA219 existente; confirmar quantidade e faixa de medição do módulo.
- Escolher o modelo de ventilador DC 12 V e obter corrente nominal, corrente de partida e datasheet.
- Dimensionar fontes, conversores, MOSFETs, proteção, fios e conectores com base nos valores reais.
- Desenhar e revisar o esquema elétrico final; validar pinos e níveis lógicos.
- Definir intervalo de amostragem, faixas plausíveis, limite local e limite remoto inicial.
- Congelar contrato MQTT v1, incluindo catálogo de erros, QoS, retain, Last Will e payloads.
- Definir parâmetros de autenticação, perfis e armazenamento seguro de credenciais.
- Executar e registrar evidências dos quatro testes R10; implementar e demonstrar R17 e R18.

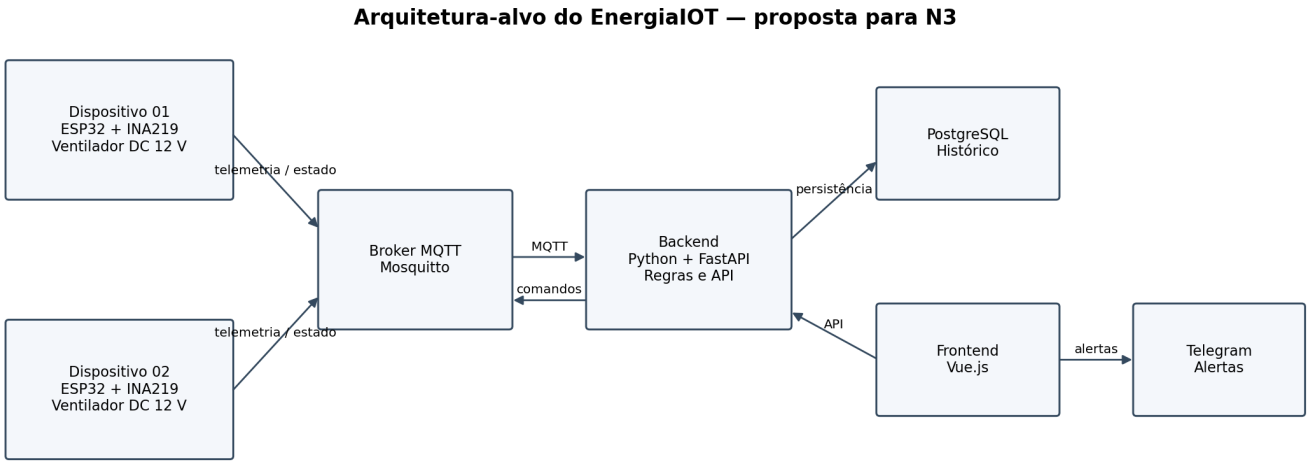
Observação de status

Este documento é uma proposta de planejamento baseada nas informações fornecidas pelo grupo e nos requisitos da disciplina. A arquitetura, tecnologias e estratégias são propostas; não significam que as funcionalidades já estejam

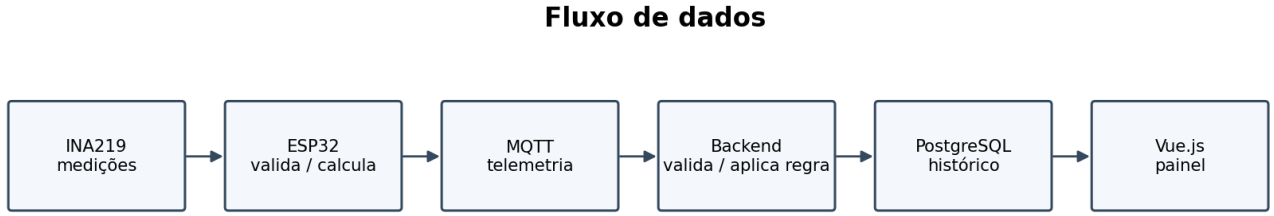
implementadas. Valores elétricos não foram inventados: permanecem pendentes até a seleção e consulta dos datasheets dos componentes.

ANEXO VISUAL — DIAGRAMAS DO PROJETO

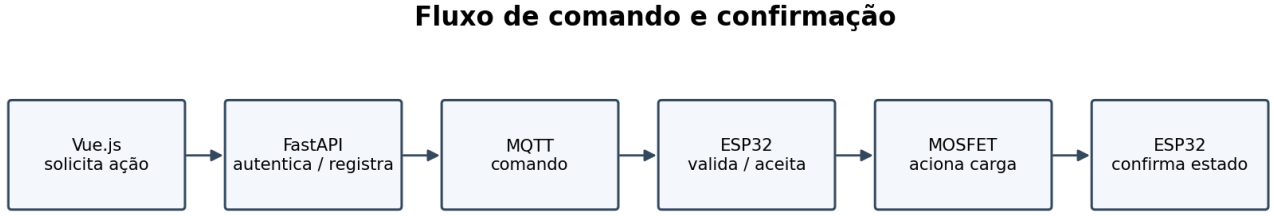
Arquitetura-alvo do EnergiaIOT



Fluxo de dados



Fluxo de comando e confirmação



Fluxograma conceitual do firmware

Estados principais do firmware (desenho conceitual)

